
Évaluation et maximisation de la robustesse d'un réseau routier

CODE SOURCE ET ANNEXES

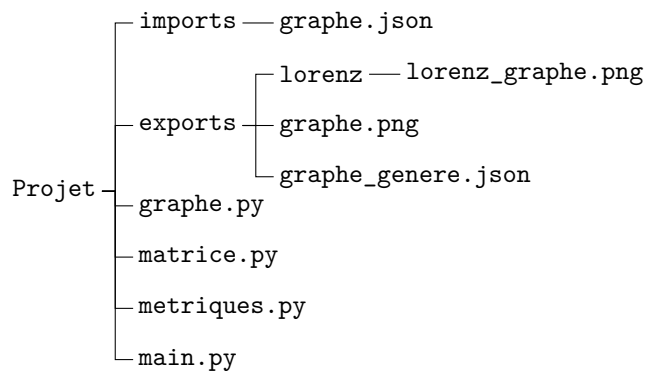
Corentin R. - Candidat XXXXX

Sommaire

1	Code source	1
1.1	Arborescence du projet	1
1.2	metriques.py	1
1.3	graphe.py	5
1.4	matrice.py	10
1.5	main.py	11
2	Annexes	12
2.1	Démonstrations	12
2.1.1	Décomposition d'une permutation en produit de cycles à supports disjoints	12
2.1.2	$\sum_{k=0}^{+\infty} A^k = (I_n - A)^{-1}$	12
2.1.3	Bonne définition de la métrique $\mathcal{C}$	13
2.2	Compléments	13
2.2.1	<code>np.linalg.det</code> : calcul par décomposition LU	13
2.2.2	Complexités des fonctions	14
3	Références bibliographiques	15

1 Code source

1.1 Arborescence du projet



1.2 metriques.py

Fonctions de calcul des métriques

```
1  #----- IMPORTS -----#
2
3
4  import graphe
5  import matrice
6
7  import matplotlib.pyplot as plt
8  import numpy as np
9
10
11
12
13
14 #----- CALCULS -----#
15
16
17 def beta_defaut(A):
18     """
19     Renvoie 1/w_moy pour la matrice d'adjacence A
20     """
21     somme = 0
22     nb = 0
23     n = len(A)
24     for i in range(n):
25         for j in range(i, n):
26             if not np.isinf(A[i, j]):
27                 nb += 1
28                 somme += A[i, j]
29     return nb/somme
30
```

```

31
32 def quicksort(l):
33     """
34     Applique l'algorithme de tri rapide à une liste
35     """
36     n = len(l)
37
38     if (n == 0 or n == 1):
39         return l
40
41     pivot = l[n//2]
42
43     gauche = [e for e in l if e < pivot]
44     milieu = [e for e in l if e == pivot]
45     droite = [e for e in l if e > pivot]
46
47     return quicksort(gauche) + milieu + quicksort(droite)
48
49
50
51
52
53 #----- METRIQUES -----#
54
55
56 def centralite_non_normalisee(Delta, c, det_I_moins_Delta):
57     """
58     Renvoie la valeur de la centralité non normalisée du cycle c dans G
59     ↪ de matrice d'adjacence Delta
60     On passe en argument det(I_n - Delta)
61     """
62     Delta_c = np.copy(Delta)
63     I = matrice.id(Delta.shape[0])
64     n = len(c)
65     for i in range(n-1):
66         Delta_c[c[i]][c[i+1]] = 0.
67         Delta_c[c[i+1]][c[i]] = 0.
68     Delta_c[c[-1]][c[0]] = 0.
69     Delta_c[c[0]][c[-1]] = 0.
70
71     det_I_moins_Delta_c = matrice.det(I - Delta_c)
72
73     if np.isclose(det_I_moins_Delta, 0.0):
74         return 0.0
75
76     return max(0.0, 1.0 - (det_I_moins_Delta_c / det_I_moins_Delta))
77
78
79
80

```

```

81 def centralites_normalisees(Delta, cycles=None):
82     """
83     Renvoie la liste des centralités normalisées des cycles
84     ↪ fondamentaux du graphe de matrice d'adjacence Delta
85     """
86     if cycles == None:
87         cycles = graphe.liste_cycles_fondamentaux(Delta)
88     if len(cycles) == 0:
89         return []
90     det_I_moins_Delta = np.linalg.det(matrice.id(Delta.shape[0]) - Delta)
91     centralites_non_norm = []
92     somme_centralites = 0
93     for cycle in cycles:
94         centralite = centralite_non_normalisee(Delta, cycle,
95             ↪ det_I_moins_Delta)
96         centralites_non_norm.append(centralite)
97         somme_centralites += centralite
98     if np.isclose(somme_centralites, 0.0):
99         return [1 / len(centralites_non_norm)] *
100             ↪ len(centralites_non_norm)
101     return [(1/somme_centralites)*c for c in centralites_non_norm]
102
103 def lorenz(Delta, filename="lorenz"):
104     """
105     Trace la courbe de Lorenz du graphe de matrice d'adjacence Delta
106     Renvoie x et y les coordonnées des points du graphe, nécessaires
107     ↪ pour calculer Gini
108     """
109     cycles = graphe.liste_cycles_fondamentaux(Delta)
110     centralites = centralites_normalisees(Delta, cycles)
111     n = len(centralites)
112
113     if n == 0:
114         x = [0.0, 1.0]
115         y = [0.0, 1.0]
116     else:
117         centralites_triees = quicksort([max(0.0, c) for c in
118             ↪ centralites])
119         y = [0.0]
120         for c in centralites_triees:
121             y.append(y[-1] + c)
122
123         total = y[-1]
124         y = [val / total for val in y]
125         x = [i / n for i in range(n+1)]
126
127     fig, ax = plt.subplots(figsize=(10, 7))
128     ax.set_aspect("equal", "box")
129     ax.plot(x, y, marker='.', color='purple')
130     ax.plot([0,1], [0, 1], ":g")

```

```

127
128     ax.set_xlabel(r'Proportion de cycles considérés')
129     ax.set_ylabel(r'Somme des centralités')
130     ax.set_title("Courbe de Lorenz")
131     ax.set_facecolor("#fafafa")
132     ax.grid(True)
133
134     fname = f"exports/lorenz/{filename}.png"
135
136     plt.savefig(fname, bbox_inches='tight', facecolor="#fafafa")
137     plt.close(fig)
138     return x, y
139
140
141 def gini(x, y):
142     """
143     Renvoie l'indice de Gini du graphe duquel on a calculé x et y dans
144     ↪    lorenz
145     """
146     if len(x) < 3 or len(y) < 3:
147         return 0.0
148     g = 0
149     for i in range(1, len(x)-1):
150         g += (x[i+1]-x[i])*(y[i] + 0.5*(abs(y[i+1]-y[i])))
151     return 1 - 2*g

```

1.3 graphe.py

Fonctions relatives à la manipulation de graphes

```
1  #-----          IMPORTS          -----#
2
3
4  import metriques
5
6  import json
7  import networkx as nx
8  import matplotlib.pyplot as plt
9  import numpy as np
10 import random
11 from collections import deque
12
13
14
15
16
17 #-----          FONCTIONS  GRAPHE          -----#
18
19
20 def from_json(fichier):
21     """
22     Convertit un fichier JSON en graphe NetworkX
23     Le fichier doit être dans le dossier imports, sans l'extension
24     ↪ ".json"
25     """
26     with open(f"imports/{fichier}.json", "r", encoding="utf-8") as f:
27         data = json.load(f)
28
29     G = nx.Graph()
30
31     for arete in data.get("aretes", []):
32         G.add_edge(
33             arete["from"],
34             arete["to"],
35             w=arete.get("w", 1.0),
36         )
37
38     return G
39
40 def to_json(G, nom):
41     """
42     Convertit un graphe NetworkX en fichier JSON
43     Le fichier sera dans le dossier exports
44     """
45     data = {
46         "noeuds": list(G.nodes()),
47         "aretes": [
```

```

48         {"from": u, "to": v, "w": attrs.get("w", 1.0)}
49     for u, v, attrs in G.edges(data=True)
50 ],
51 }
52
53 with open(f"exports/{nom}.json", "w", encoding="utf-8") as f:
54     json.dump(data, f, indent=4, ensure_ascii=False)
55
56
57 def to_img(G, filename="graphe", titre="", taille=(10, 7)):
58     """
59     Exporte un graphe en image
60     Le fichier sera dans le dossier exports
61     """
62     fig, ax = plt.subplots(figsize=taille)
63     pos = nx.spring_layout(G, seed=42)
64
65     nx.draw_networkx_nodes(G, pos, ax=ax, node_color="#4C72B0",
66         ↪ node_size=600)
67     nx.draw_networkx_labels(G, pos, ax=ax, font_color="white",
68         ↪ font_weight="bold")
69     nx.draw_networkx_edges(G, pos, ax=ax, width=1.5, alpha=0.7)
70
71     poids_labels = {(u, v): f"{attrs.get('w', 1.0)}" for u, v, attrs in
72         ↪ G.edges(data=True)}
73     nx.draw_networkx_edge_labels(G, pos, edge_labels=poids_labels, ax=ax,
74         ↪ font_size=9, bbox=dict(color='#fafafa'))
75
76     if titre != "":
77         ax.set_title(titre, fontsize=14, fontweight="bold")
78     ax.axis("off")
79     plt.tight_layout()
80     plt.savefig(f"exports/{filename}.png", facecolor="#fafafa", dpi=150,
81         ↪ bbox_inches="tight")
82     plt.close(fig)
83
84 def matrice_adjacence(G, etat=1, beta=1):
85     """
86     Renvoie la matrice d'adjacence de G
87     - Si etat = 1 : matrice pondérée par  $e^{-\beta w_{ij}}$ 
88     - Si etat = 2 : matrice non pondérée (1 si l'arête existe, 0 sinon)
89     - Sinon, matrice pondérée par  $w_{ij}$ 
90     """
91     noeuds = list(G.nodes())
92     n = len(noeuds)
93     index = {noeud: i for i, noeud in enumerate(noeuds)}
94
95     if etat == 1 or etat == 2:
96         matrice = np.zeros((n, n), dtype=float)
97     else:

```

```

94     matrice = np.full((n, n), fill_value=np.inf, dtype=float)
95
96     for u, v, attrs in G.edges(data=True):
97         poids = attrs.get("w", 1.0)
98         i, j = index[u], index[v]
99         if etat == 1:
100             poids = np.exp(-(beta * poids))
101         elif etat == 2:
102             poids = 1.0
103         matrice[i, j] = poids
104         matrice[j, i] = poids
105
106     return matrice
107
108
109 def cycles_fondamentaux_sommet(Delta, s):
110     """
111     Renvoie la liste des cycles fondamentaux de G de matrice
112     ↪ d'adjacences Delta à partir du sommet s
113     L'arbre est construit par DFS
114     """
115     n = len(Delta)
116
117     # 1. Arbre couvrant
118     racine = s
119     parent = {i: None for i in range(n)}
120     profondeur = {i: 0 for i in range(n)}
121     visites = {racine}
122     file = deque([racine])
123     aretes_arbre = []
124
125     while file:
126         i = file.popleft()
127         for j in range(n):
128             if not np.isinf(Delta[i, j]) and Delta[i, j] != 0 and j not
129                 ↪ in visites:
130                 visites.add(j)
131                 parent[j] = i
132                 profondeur[j] = profondeur[i] + 1
133                 file.append(j)
134                 aretes_arbre.append(tuple(metriques.quicksort((i, j))))
135
136     # 2. Détermination des arêtes de G qui ne sont pas dans l'arbre
137     aretes_hors_arbre = []
138     set_arbre = set(aretes_arbre)
139     for i in range(n):
140         for j in range(i+1, n):
141             if not np.isinf(Delta[i, j]) and Delta[i, j] != 0.:
142                 if tuple(metriques.quicksort((i, j))) not in set_arbre:
143                     aretes_hors_arbre.append((i, j))

```

```

143     # 3. Détermination des cycles pour chaque arête hors de l'arbre
144     cycles_fonda = []
145     for (i,j) in aretes_hors_arbre:
146         path_i = [i]
147         curr = i
148         while parent[curr] is not None:
149             curr = parent[curr]
150             path_i.append(curr)
151
152         path_j = [j]
153         curr = j
154         while parent[curr] is not None:
155             curr = parent[curr]
156             path_j.append(curr)
157
158         ancetre_commun = None
159         for node in path_i:
160             if node in path_j:
161                 ancetre_commun = node
162                 break
163
164         partie_i = path_i[:path_i.index(ancetre_commun) + 1]
165         partie_j = path_j[:path_j.index(ancetre_commun) + 1]
166
167         partie_j.reverse()
168
169         cycle = partie_i + partie_j
170         cycles_fonda.append(cycle)
171
172     return cycles_fonda
173
174
175 def liste_cycles_fondamentaux(Delta):
176     """
177     Renvoie la liste des cycles fondamentaux de G de matrice
178     ↪ d'adjacences Delta (ensemble des cycles à partir de tous les
179     ↪ sommets)
180     """
181     n = len(Delta)
182     liste = []
183     for i in range(0, n-1):
184         liste.append(cycles_fondamentaux_sommet(Delta, i))
185
186
187 def generer(n=20, d=0.3, filename="graphe"):
188     """
189     Génère un graphe à n sommets, une densité d'arêtes de d, et
190     ↪ l'enregistre en json
191     Dans filename, ne pas préciser le .json
192     """
193     edges = []

```

```

191     edge_set = set()
192
193     nodes = list(range(n))
194     random.shuffle(nodes)
195     for i in range(1, n):
196         s = nodes[i]
197         t = nodes[random.randint(0, i-1)]
198         edges.append({"from": s, "to": t, "w": random.randint(1, 20)})
199         edge_set.add((min(s, t), max(s, t)))
200
201     for s in range(n):
202         for t in range(s + 1, n):
203             if (s, t) in edge_set:
204                 continue
205             if random.random() < d:
206                 edges.append({"from": s, "to": t, "w": random.randint(1,
207                                     ↪ 20)})
208                 edge_set.add((s, t))
209
210     with open(f"./exports/exemples/{filename}.json", "w",
211             ↪ encoding="utf-8") as f:
212         json.dump({"noeuds": list(range(n)), "aretes": edges}, f,
213                 ↪ indent=4)
214
215
216 #----- ALIAS DE FONCTIONS NX -----#
217
218 def nouveau():
219     """Renvoie un graphe non orienté pondéré vide"""
220     return nx.Graph()
221
222
223 def ajouter_arete(G, s, t, w):
224     """
225     Ajoute l'arête (s, t) à G avec la pondération w, modifie la
226     ↪ pondération si elle existe déjà
227     """
228     if G.has_edge(s, t):
229         G[s][t]["w"] = w
230     else:
231         G.add_edge(s, t, w=w)
232
233 def supprimer_arete(G, s, t):
234     """
235     Supprime l'arête (s, t) de G
236     """
237     if not G.has_edge(s, t):

```

```

238         raise Exception("supprimer_arete : arête inexistante")
239     G.remove_edge(s, t)
240
241
242 def sommets(G):
243     """
244     Renvoie la liste triée des sommets de G
245     """
246     try:
247         return metriques.quicksort(G.nodes())
248     except TypeError:
249         return list(G.nodes())

```

1.4 matrice.py

Fonctions relatives à la manipulation de matrices

```

1  #----- IMPORTS -----#
2
3
4  import numpy as np
5
6
7
8
9
10 #----- FONCTIONS -----#
11
12
13 def det(M):
14     """
15     Calcule le déterminant de M (par décomposition LU)
16     """
17     if M.ndim != 2 or M.shape[0] != M.shape[1]:
18         raise Exception("det : matrice non carrée")
19     return np.linalg.det(M)
20
21
22 def id(n):
23     return np.eye(n, dtype=float)

```

1.5 main.py

Génération de 10 graphes aléatoires, calcul de leur courbe de Lorenz et indice de Gini

```
1  import graphe
2  import metriques
3
4  import random
5  import time
6
7
8
9  start = time.time()
10
11  for i in range(1, 11):
12
13      print(f"Génération du graphe {i}")
14
15      n = random.randint(5, 30)
16      d = random.randint(15, 40) / 100
17
18      graphe.generer(n, d, f"graphe_{i}")
19
20      print(f"Importation du graphe {i}")
21
22      g = graphe.from_json(f"../exports/exemples/graphe_{i}")
23
24      print(f"Calculs sur le graphe {i}")
25
26      Delta_temp = graphe.matrice_adjacence(g)
27      beta = metriques.beta_defaut(Delta_temp)
28
29      Delta = graphe.matrice_adjacence(g, 1, beta)
30
31      graphe.to_img(g, f"graphe_{i}")
32
33      x, y = metriques.lorenz(Delta, f"lorenz_{i}")
34      print(f"GINI {i} : {metriques.gini(x, y)}")
35
36  end = time.time()
37
38  print(f"Durée d'exécution : {end-start} secondes ({(end-start)/60}
    ↪ minutes)")
```

2 Annexes

2.1 Démonstrations

2.1.1 Décomposition d'une permutation en produit de cycles à supports disjoints

Toute permutation $\sigma \in S_n$ se décompose de façon unique (à ordre des facteurs près) en un produit de cycles à supports disjoints

Existence : par récurrence forte

$n = 1$: la seule permutation de S_1 est *id*. C'est un cycle.

$n \in \mathbb{N}^*$: supposons le résultat vérifié pour tout $i \in E = \llbracket 1; n \rrbracket$. Soit $\sigma \in S_{n+1}$.

Soit $a \in E$. Soit p le plus petit entier tel que l'on retombe sur un élément déjà vu lorsqu'on parcourt $a, \sigma(a), \sigma^2(a), \dots$.

$\exists q \in \llbracket 0; p-1 \rrbracket, \sigma^q(a) = \sigma^p(a)$.

Si $q \neq 0$, alors $\sigma(\sigma^{p-1}(a)) = \sigma(\sigma^{q-1}(a))$ donc $\sigma^{p-1}(a) = \sigma^{q-1}(a)$, ce qui contredit la minimalité de p . Donc $q = 0$

On obtient alors un cycle $(a \ \sigma(a) \ \sigma^2(a) \ \dots \ \sigma^{p-1}(a))$.

Soit $E_1 = \{a, \sigma(a), \dots, \sigma^{p-1}(a)\}$, $E_2 = E \setminus E_1$.

Si $E_2 \neq \emptyset$, on peut décomposer la restriction de σ à E_2 en produit de cycles à supports disjoints par hypothèse.

On décompose ainsi σ en produit de cycles à supports disjoints, d'où l'existence.

Unicité :

Soit $\sigma \in S_n$. Soit $c_1 \circ c_2 \circ \dots \circ c_a$ et $c'_1 \circ c'_2 \circ \dots \circ c'_b$ deux décompositions de σ en cycles à supports disjoints.

On peut supposer (quitte à réordonner les cycles) que $1 \in \text{supp}(c_1) \cap \text{supp}(c'_1)$.

Alors $c_1^2(1) = c'_1{}^2(1)$, $c_1^3(1) = c'_1{}^3(1)$, et donc $c_1 = c'_1$.

Donc $c_2 \circ \dots \circ c_a = c'_2 \circ \dots \circ c'_b$.

On applique le même procédé aux nouveaux premiers cycles jusqu'à obtenir $c_a = c'_b$, et donc les deux décompositions sont égales. D'où l'unicité.

2.1.2 $\sum_{k=0}^{+\infty} A^k = (I_n - A)^{-1}$

$$\forall A \in \mathcal{M}_n(\mathbb{R}), \rho(A) < 1 \implies \sum_{k=0}^{+\infty} A^k = (I_n - A)^{-1}$$

Convergence :

Soit $A \in \mathcal{M}_n(\mathbb{R})$ telle que $\rho(A) < 1$.

On admet que $\forall \varepsilon > 0, \exists \|\cdot\|_\varepsilon$ norme sous-multiplicative, $\forall A \in \mathcal{M}_n(\mathbb{R}), \|A\|_\varepsilon \leq \rho(A) + \varepsilon$

On prend ε tel que $\rho(A) + \varepsilon < 1$, et $\|\cdot\|_\varepsilon$ correspondant.

Par sous-multiplicativité, $\forall k \in \mathbb{N}, \|A^k\|_\varepsilon \leq \|A\|_\varepsilon^k \leq (\rho(A) + \varepsilon)^k$.

Or $0 < \rho(A) + \varepsilon < 1$, donc $\sum (\rho(A) + \varepsilon)^k$ converge.

Donc $\sum A^k$ converge absolument donc converge.

Somme :

$$\forall N \in \mathbb{N}, (I_n - A) \sum_{k=0}^N A^k = \sum_{k=0}^N A^k - \sum_{k=1}^{N+1} A^k = I_n - A^{N+1} \xrightarrow{N \rightarrow +\infty} I_n \text{ par convergence}$$

$$\text{De même, } \left(\sum_{k=0}^N A^k \right) (I_n - A) \xrightarrow{N \rightarrow +\infty} I_n$$

D'où le résultat.

Remarque: on utilise cette propriété sur Δ_c , dont les coefficients sont des $e^{-\beta w_{ij}}$ avec $w_{ij} \neq 0$, donc tous les coefficients de Δ_c sont dans $]0; 1[$. Ainsi, $\rho(\Delta_c) < 1$.

2.1.3 Bonne définition de la métrique $\mathcal{C}$

$$\mathcal{C}: \begin{cases} \mathbf{C} \longrightarrow \mathbb{R}_+^* \\ c \longmapsto \frac{1}{\sum_{d \in \mathbf{C}} \mathcal{C}(d)} \underbrace{\left(1 - \frac{\det(I_{|\mathbf{S}|} - \Delta_c)}{\det(I_{|\mathbf{S}|} - \Delta)} \right)}_{\mathcal{C}(c)} \end{cases}$$

Existence :

Puisque $I_{|\mathbf{S}|} - \Delta_c$, et donc $I_{|\mathbf{S}|} - \Delta$ sont inversibles, $\det(I_{|\mathbf{S}|} - \Delta) \neq 0$.

Donc pour tout cycle c , $\frac{\det(I_{|\mathbf{S}|} - \Delta_c)}{\det(I_{|\mathbf{S}|} - \Delta)}$ existe.

Positivité :

On a $\text{Sp}(I_{|\mathbf{S}|} - \Delta) = \{1 - \lambda \mid \lambda \in \text{Sp}(\Delta)\}$

Or $\rho(\Delta) < 1$ et $\forall \lambda \in \text{Sp}(\Delta), |\lambda| < \rho(\Delta)$, donc $\text{Sp}(I_{|\mathbf{S}|} - \Delta) \subset \mathcal{D}(0, 1)$

$$\det(I_{|\mathbf{S}|} - \Delta) = \prod_{\lambda \in \text{Sp}(\Delta)} (1 - \lambda)$$

Or $\forall \mu \in \text{Sp}(I_{|\mathbf{S}|} - \Delta)$, $\Re(\mu) > 0$, et les éventuelles valeurs propres complexes de Δ apparaissent par paire de conjugués, dont le produit est donc un réel strictement positif.

Ainsi, $\det(I_{|\mathbf{S}|} - \Delta) > 0$. De même, $\det(I_{|\mathbf{S}|} - \Delta_c) > 0$.

Donc $\mathcal{C} > 0$.

2.2 Compléments

2.2.1 `np.linalg.det` : calcul par décomposition LU

Afin de ne pas trop perdre en complexité lors du calcul des déterminants dans $\mathcal{C}$, la fonction `np.linalg.det` du module NumPy est utilisée. Elle calcule le déterminant d'une matrice A par décomposition LU :

$$A = P^{-1} \underbrace{\begin{pmatrix} 1 & 0 & \cdots & 0 \\ * & \ddots & & \vdots \\ \vdots & & \ddots & 0 \\ * & \cdots & * & 1 \end{pmatrix}}_L \underbrace{\begin{pmatrix} * & 0 & \cdots & 0 \\ \vdots & \ddots & & \vdots \\ \vdots & & \ddots & 0 \\ * & \cdots & \cdots & * \end{pmatrix}}_U$$

c'est-à-dire $A = P^{-1}LU$ avec :

- P^{-1} matrice de permutation (coefficients dans $\{0; 1\}$, un seul 1 par ligne et par colonne)
- L triangulaire inférieure avec uniquement des 1 sur la diagonale
- U triangulaire supérieure

En passant au déterminant :

$$\det(A) = \det(P^{-1}) \det(L) \det(U) = \det(P) \det(U) = \pm \prod_{i=1}^n U_{i,i}$$

Cette décomposition est construite par pivot de Gauss (avec échange de ligne ou de colonne en cas de pivot nul, d'où la matrice P), et existe toujours pour une matrice inversible.

Complexité du calcul du déterminant :

- Factorisation LU : nécessite une élimination de Gauss, qui demande environ $\frac{2}{3}n^3$ opérations flottantes $\rightarrow O(n^3)$

- Calcul de $\det(P)$: $\det(P)$ est égal à la signature de la permutation de P $(-1)^{\text{Inv}(P)}$ avec $\text{Inv}(P)$ le nombre d'inversions de lignes et de colonnes faites lors du calcul de la factorisation, qu'on peut stocker au fur et à mesure du calcul $\rightarrow O(1)$
- Calcul de $\det(U)$: produit de n coefficients accessibles en temps constant $\rightarrow O(n)$

On passe ainsi d'un calcul de déterminant en $O(n!)$ (méthode naïve) à un calcul en $O(n^3)$.

2.2.2 Complexités des fonctions

graphe.py

Fonction	Complexité	Commentaire
from_json	$O(A)$	
to_json	$O(A)$	
to_img	$O(S ^3)$	
matrice_adjacence	$O(S ^2 + A)$	Initialisation $O(S ^2)$, remplissage $O(A)$
cycles_fondamentaux_sommet	$O(S ^2 + A)$	BFS $O(S + A)$, boucles $O(S ^2)$ et $O(A)$
liste_cycles_fondamentaux	$O(S ^3)$	
generer	$O(n^2)$	En général, $n = S $
nouveau	$O(1)$	
ajouter_aretes	$O(1)$	
supprimer_aretes	$O(1)$	
sommets	$O(S \log S)$	<code>G.nodes()</code> $O(1)$ + quicksort

matrice.py

Fonction	Complexité	Commentaire
diff	$O(S ^2)$	
det	$O(S ^3)$	
id	$O(S ^2)$	

metriques.py

Fonction	Complexité	Commentaire
beta_default	$O(S ^2)$	
quicksort	$O(n \log(n))$	En moyenne, pire cas en $O(n^2)$
centralite_non_normalisee	$O(S ^3)$	Calcul du déterminant
centralites_normalisees	$O(n S ^3)$	$n = \text{nb_cycles} = A - S + 1$ ici
lorenz	$O(S \log(S) + n S ^3)$	<code>centralites_normalisees</code> + quicksort + courbe avec n points
gini	$O(n)$	

3 Références bibliographiques

- [1] Vincenzo Petito, Maurizio Leotta et Marina Ribaud : *Improving the performance of road network analysis: the Morandi Bridge case study*. <https://doi.org/10.5220/0007745702590266> (2019)
- [2] Pierre-Louis Giscard et Yohan Hosten : *Peut-on quantifier l'importance du périphérique parisien ?* <https://inria.hal.science/hal-05372993/> (2025)
- [3] Pauline Gauthier, Angelo Funo et Nour-Eddin El Faouzi : *Network resilience: how to identify critical links subject to day-to-day disruption*. <https://doi.org/10.1177/0361198118792115> (2018)
- [4] Max O. Lorenz : *Methods of measuring the concentration of wealth*. American Statistical Association (1905)
- [5] Corrado Gini : *Measurement of inequality of income*. Economic Journal 31 (1921)
- [6] Keith Paton : *An algorithm for finding a fundamental set of cycles of a graph*. <https://doi.org/10.1145/363219.363232> (1969)
- [7] François De Marçay : *Décomposition $A = LU$ de matrice A quelconque*. <https://www.imo.universite-paris-saclay.fr/~joel.merker/Enseignement/Algebre-Lineaire-Geometrie/lu.pdf> (2021)